

Rumi Protocol Security Review

(June 2026)

Date: 2026-06-09 **Commit reviewed:**

[e49ed103f49c065807f58eb7b034e37eaffc7b96](#) (branch [main](#)), clean working tree at review start. **Reviewer:** [audit-icp-cdp](#) harness, 16 specialist finder passes, 3 differential passes, adversarial two-lens verification of every Medium-and-above finding, then 5 adversarial self-review passes over the fixes. **Prior reviews folded in:** the April 2026 combined internal + AVAI review (published), the 2026-06-03 backend and cross-chain audits, and the **2026-06-05 audit that was completed but never published** (its findings are incorporated below).

The working tree now contains the fixes for every finding marked "fixed this session." The commit named above ([e49ed10](#)) is the pre-fix state. Every fix carries a regression test; the full suite is green (backend 370 unit + 30 PocketIC, all six satellite crates, 115 frontend vitest).

Executive summary

This review covered the entire Rumi Protocol stack at [e49ed10](#) : the core CDP backend, the 3pool StableSwap (3USD token), the pair AMM, the stability pool, the treasury, the liquidation bot, the now-live points/airdrop engine, the analytics canister, the experimental cross-chain module, and the vault frontend service layer. It ran in differential mode against the prior 2026-06-05 audit, whose remediations had merged to [main](#) (PR #228) and partly deployed but had not been published.

The headline result is three new HIGH-severity findings, all in the **redemption mechanism**, and all rooted in the same gap: the per-vault liquidation lock the 2026-06-05 audit introduced (to stop two liquidators from over-seizing one vault) was honored by the liquidation paths but **not** by redemption or by ordinary owner write-ops. A redeemer or a depositor's own repay/withdraw could therefore interleave with a liquidation on the same vault and either drain collateral from other users' vaults through the shared collateral account, or trap a balance-changing operation after its transfer had already settled. A fourth HIGH-class issue (a redemption paying out more collateral than any vault actually gave up) compounded the first. None of these required privileged access; redemption and the vault write-ops are public endpoints.

All three HIGH findings are fixed, proven, and tested this session. So are five MEDIUM and twelve LOW findings spread across every canister. Two informational items document pre-existing conditions surfaced during the review.

The prior 2026-06-05 audit's own findings (four HIGH: the per-vault liquidation lock itself, the oracle freshness fail-open for non-ICP collateral, the debt-ceiling race, and the unchunked points epoch close; plus the satellite and stability-pool work) are confirmed present and holding at [e49ed10](#), with one exception that this review re-opened and closed: the liquidation bot's claim path had been missed by the freshness retrofit.

- **New findings this review: 0 Critical, 3 High, 8 Medium, 12 Low, 2 Informational.**
- **Fixed this session: 3 High, 8 Medium, 11 Low** (the 12th Low is the cross-chain recovery gap, which stays open at its dev-gated severity inside the parked experimental module).
- **Prior 2026-06-05 audit: 39 findings verified fixed and holding, 0 regressed.**

No fund losses, no protocol pauses, no emergency interventions occurred during this review. The protocol ran continuously on mainnet throughout.

How this review fits the prior work

The April 2026 combined review closed 97 findings (the internal three-pass review plus the AVAI external pre-audit). Through May and early June a series of follow-up audits (2026-06-03 backend, 2026-06-03 cross-chain, 2026-06-05 full) found and fixed a second generation of issues, the most important being a class of async-state races around liquidation. Those 2026-06-05 fixes were the right fixes for the axis they targeted (two liquidators racing one vault), and they hold up.

What this review found is that the same underlying hazard, a vault mutated across an [await](#) by a concurrent caller, had **two more axes** the prior fix did not cover: redemption, and the vault owner's own write operations. The prior fix put a per-vault lock on the liquidation entry points. This review extends that lock to every operation that mutates a vault across an inter-canister call, and teaches the redemption water-fill to skip a vault that any such operation (or a bot claim) is mid-flight on. That is the spine of the three HIGH fixes.

This is the value of differential re-auditing: a fix that is correct for the case it was written for can still leave a sibling case open, and only a fresh sweep against the whole surface catches it.

High findings

AR-B-001: Redemption bypasses the per-vault liquidation and bot-claim exclusion

The 2026-06-05 audit added `VaultLiquidationGuard`, a per-vault lock so two liquidators cannot both be paid the same vault's collateral out of the protocol's single shared collateral account. It also relies on the `bot_processing` flag, which the liquidation bot sets when it claims a vault (taking the collateral) but before it confirms (writing down the debt). Both mechanisms were honored by the liquidation entry points. **Redemption honored neither.**

The redemption water-fill selects the lowest-CR vault, which is exactly the vault a liquidation or a bot claim is most likely to be working on. Three concrete harms followed: a redemption landing during a manual or stability-pool liquidation paid both the liquidator (from a stale pre-await snapshot) and the redeemer from the shared pool; a redemption landing in the bot's claim-to-confirm window seized collateral the bot had already been paid; and the bot's confirm step used a non-saturating subtraction that **trapped** if a redemption had reduced the vault's debt in between, leaving the vault permanently stuck with the bot's collateral gone.

Fix. The redemption water-fill now skips any vault with `bot_processing == true` or under the per-vault lock (`is_vault_liquidating`), mirroring the `check_vaults` skip. Every liquidation payout is re-capped to the post-await collateral the vault actually gave up, so a residual race degrades to a smaller payout rather than a shared-pool drain. The bot confirm and the admin stuck-claim resolver now use saturating subtraction, so they can never trap on a shrunken vault. Event replay stays exact because the recorded liquidation/redemption events carry the actual applied amounts, which replay re-applies directly.

AR-B-003: Owner write-ops not serialized against a concurrent liquidation or redemption

The per-vault lock excluded two liquidations of one vault. It did **not** exclude a concurrent owner write-op (repay, partial-withdraw, borrow, add-margin, close) on the same vault. Those operations validated a snapshot of the vault, performed an inter-canister transfer (an `await`), then committed using non-saturating asserting arithmetic with no re-check. A liquidation or redemption committing first falsified the snapshot, and the owner op's commit then **trapped after its transfer had already settled**: a partial-withdraw left the user holding

collateral the vault books were never debited for (phantom collateral, payable again on a later seizure); a repay pulled the user's icUSD with no debt credit. The reverse ordering let a liquidation over-seize from the shared pool.

Fix. Every owner write-op that spans an `await` now acquires the same per-vault lock (`VaultLiquidationGuard`), so a liquidation or redemption cannot interleave with it on one vault. Independently, the asserting commit sites (`remove_margin_from_vault`, `repay_to_vault`) were changed to saturating clamps, so even an unanticipated interleaving degrades to under-application instead of a trap-after-transfer. The self-review caught one endpoint (`partial_repay_to_vault`) initially missed by this rollout; it is now covered, and the regression fence asserts the lock on all twelve write-ops.

RED-001: Redemption payout not clamped to the collateral actually seized

The redeemer's payout was computed as `claim / price` using the full post-fee claim, independent of how much collateral the water-fill actually removed. When the claim exceeded the priority collateral's total redeemable vault debt (or hit an underwater vault), the protocol burned the full claim but seized less collateral than it paid out. The difference came from the single shared collateral account, draining the backing of co-collateral vaults that were never redeemed. The protocol's total-collateral-ratio solvency latch could not see this, because it reads vault collateral state, which the water-fill leaves unchanged for the drained vaults.

Fix. The payout is now derived from the icUSD the water-fill actually consumed, never from the requested claim. `redeem_collateral` rejects up front any claim larger than the redeemable vault debt for the target collateral, and any unconsumed remainder of the claim is refunded to the redeemer through the same durable-refund saga the reserve-redemption path already uses. Fund conservation was verified algebraically and by test: total value out (collateral seized plus icUSD refunded) equals the post-fee amount burned, and the refund is additionally hard-capped at that amount.

Medium findings (all fixed this session)

- **RED-002 (redemption / oracle).** `redeem_collateral` validated price freshness, priced the dynamic fee, and bumped the rising-fee defense against the caller-supplied collateral type, but seized the redemption-priority winner, which can be a different type. A redeemer could pass a deep-debt type to floor the anti-mass-redemption fee while draining a thin type against

a price with no staleness gate. The endpoint now resolves the priority winner up front and keys freshness, fee, and the base-rate bump on it, matching the already-correct reserve-spillover path.

- **ORC-001 (oracle).** `bot_claim_liquidation` was the one liquidation entry the non-ICP freshness retrofit (VER-001 from the prior audit) missed: it gated only on the ICP price age, not on the vault collateral's own staleness ceiling, and it ignored the admin liquidation-freeze brake. Dormant while the bot's allowlist is ICP-only; it becomes HIGH the moment any non-ICP collateral is added. Both gates are now enforced, fixed before any non-ICP bot collateral is enabled.
- **AR-S-002 (stability pool).** The SP-102 reentrancy guard gated deposit/withdraw/claim during a liquidation, but `opt_in_collateral` / `opt_out_collateral` (which change the apportionment denominator) were not gated. A depositor opting out during the liquidation await escaped its share of the burn and left the pool's tracked aggregate above the real ledger balance. Both endpoints are now gated.
- **IC-S-001 (stability pool).** The `deposit_as_3usd` failure refund sent the gross amount with the result discarded, drifting the pool balance below tracked deposits and stranding tokens with no record if the refund itself failed. It now refunds net of fee and persists a per-user pending claim with a user-callable recovery endpoint, matching the 3pool pattern.
- **UPG-001 (liquidation bot).** On a stable-memory decode failure the bot wiped its config to default (the only canister in the stack that wiped rather than trapped), which reset its migration flag and armed a legacy raw-offset write that would corrupt the memory manager header on the next upgrade. It now traps on decode failure (preserving state for recovery) while still defaulting cleanly on a genuine fresh install, and the legacy write path can no longer re-arm.
- **PTS-002 (points, live).** The public epoch-status query exposed the open epoch's secret-seed-derived future snapshot times, defeating the commit-reveal anti-snipe design. The times are now withheld until they pass; the admin view keeps full visibility.
- **PTS-001 (points, live).** The stability-pool and AMM points pollers used an event-id cursor as an array index into bounded ring buffers; once a source log rotated, the cursor outran the trimmed buffer and registration of pool-only participants silently stalled forever. The pollers are now rotation-safe (read by position, filter by event id), matching the analytics tailer. Cursor semantics are unchanged, so no state migration is needed.

- **DOS-001 (3pool / amm).** Event-history query endpoints used a caller-supplied page size with no server-side ceiling, so a single oversized request could exceed the reply-size limit (amm) or grow unboundedly with protocol volume (3pool). Every such endpoint now clamps to a 2000-record maximum.
-

Low findings (eleven fixed this session, one open at dev-gated severity)

ID	Canister	Title	Status
IC-B-002	backend	repay/liquidation discarded the unminted interest-share return, dropping treasury revenue; now re-queued	FIXED
DBT-001	backend	interest accrual rounded down (borrower favor); now rounds up, defers on overflow	FIXED
DBT-002	backend	numeric <code>Token::Sub</code> panics; the reachable post-transfer commit sites are now saturating (via AR-B-003)	FIXED (sites)
ICRC-002	treasury	withdraw ignored the ledger fee, drifting tracked balance above holdings; now net-of-fee	FIXED
ICRC-003	treasury	withdraw <code>created_at_time</code> was live time, not request-id-derived; now persisted and reused per request	FIXED
ICRC-004	stability pool	<code>claim_collateral</code> fee fallback was 0 (over-credit); now the conservative SP-104 fallback	FIXED
ICRC-001	3pool (3USD)	the 3USD token ignored <code>created_at_time</code> (no dedup); standard ICRC-1 dedup added	FIXED
IC-S-003	3pool / amm	<code>transfer_to_user</code> could send nothing for a dust output with <code>min=0</code> ; now rejected before any state debit	FIXED
FE-001	frontend	seven vault flows used unbounded non-expiring approvals; now bounded to 30 days	FIXED
FE-002	frontend	Oisy false-negative verifier trusted a non-principal-keyed cache; now principal-keyed and cleared on wallet change	FIXED
FE-003	frontend	swap UI showed gross output while the backend enforces net; display and min bounds now net-correct per route	FIXED
XC-001	backend (chains)	verified recovery endpoints are Monad-only, fail-closed for Solana	OPEN (dev-gated, experimental module)

The XC-001 cross-chain recovery gap stays open at its dev-gated severity. The cross-chain module ([chains/](#)) is experimental and parked: its timers are off, it runs only on testnet/devnet, and it is gated behind a developer flag. It is not part of the ICP-native production path. It should be closed before that module is ever un-gated on mainnet.

Informational

- **Redemption fee LP-replay divergence (pre-existing).** The disaster-recovery event-replay path credits developer liquidity-pool shares for redemption fees that the live path never credits, and the replay-vs-live equality check does not cover that field. This is pre-existing (it is byte-identical on [main](#)), it was preserved verbatim by this review's redemption rewrite, and it only matters in a full event-log replay (not on normal stable-memory upgrades). It is documented rather than changed here, because reconciling it safely requires analyzing whether any historical redemption ever credited that account, which is out of scope for an audit changeset. Recommended as a dedicated follow-up.
 - **[self_check](#) feature did not compile (now fixed).** The on-canister replay-vs-live invariant (the backstop for redemption replay exactness) referenced a stale crate name and failed to build. It is corrected this session and should be added to CI so it stays compiling.
-

Prior-audit continuity (2026-06-05 and earlier)

Sixty-two prior findings were re-verified at [e49ed10](#) . Thirty-nine are confirmed fixed and holding, including the headline 2026-06-05 work: the per-vault liquidation lock ([VaultLiquidationGuard](#)), the oracle freshness fail-closed for non-ICP collateral, the debt-ceiling / global-mint-cap reservation guard, the chunked exactly-once points epoch close, the 3pool pending-claim recovery, the stability-pool realized-debit accounting, and the trap-not-wipe upgrade hardening across the satellites. Twenty are carried-open accepted items (documented low-severity housekeeping and the owner-accepted stability-pool deposit-before-liquidation behavior). None regressed. The full status table is in [prior-finding-statuses.md](#) .

Two prior items were reclassified or extended: the non-ICP oracle freshness ceiling, present on the manual and stability-pool liquidation paths, was found missing on the bot claim path and is now added (ORC-001 above); and the

carried-open interest-rounding and numeric-panic LOWs are now closed at their reachable sites.

Deployment state at review (measured live on mainnet)

Remediation status: the fixes for every finding in this report ship to mainnet in the coordinated June 2026 release published alongside it (the backend, stability pool, analytics, 3pool, AMM, treasury, liquidation bot, points, and both frontends). Current module hashes are verifiable on-chain at any time with `icp canister status <id> --network ic`. The table below is the snapshot measured at review time (pre-release), retained as the audit record.

Canister	Carries the 2026-06-05 audit fixes?	Notes
<code>rumi_protocol_backend</code> (<code>tfesu-...</code>)	Yes (post-PR-228)	The new HIGH fixes in this review are NOT yet deployed; gate the next backend upgrade on them.
<code>rumi_3pool</code> (<code>fohh4-...</code>)	Yes	PR #228 + the PR #230 net-fee fix are live.
<code>rumi_amm</code> (<code>ijlzs-...</code>)	Yes	PR #228 + #230 live.
<code>rumi_stability_pool</code> (<code>tmhzi-...</code>)	No (pre-PR-228)	Still runs without the SP realized-debit, reentrancy-guard, and trap-not-wipe fixes; the backend side is forward-compatible. The IC-S-001/AR-S-002/ICRC-004 fixes in this review also ride the next SP upgrade.
<code>rumi_analytics</code> (<code>dtlu2-...</code>)	No	The per-event decode fix is not yet deployed.
<code>rumi_points</code> (<code>bfnu3-...</code>)	Yes (fresh install)	Live, season 1 running. The PTS-001/PTS-002/AR-S-001 fixes here ride the next points upgrade; deploy the frontend declarations with or before that upgrade (the snapshot-time field becomes optional in the candid interface).
<code>liquidation_bot</code> (<code>nygob-...</code>)	n/a	The UPG-001 fix rides its next upgrade.
<code>vault_frontend</code> (<code>tcfua-...</code>)	partial	The Oisy and net-fee fixes are partly live; FE-001/2/3 ride the next frontend deploy.

The practical consequence: this review's three HIGH redemption fixes, plus the stability-pool and analytics fixes from the prior audit that have not yet shipped, should be bundled into the next coordinated backend + stability-pool + analytics deploy, with the points and frontend fixes following on their own cadence.

Recommended actions

1. **Gate the next backend mainnet deploy** on the three HIGH redemption fixes (AR-B-001, AR-B-003, RED-001) plus RED-002 and ORC-001. These

close live fund-loss-class issues reachable from public endpoints.

2. **Deploy the still-pending stability-pool and analytics fixes** from the 2026-06-05 audit in the same window, now joined by IC-S-001, AR-S-002, ICRC-004 (SP) and the analytics per-event decode.
3. **Points and frontend** ride their own deploys; deploy the updated frontend declarations with or before the points upgrade because of the optional-field candid change.
4. **Before enabling any non-ICP collateral for the liquidation bot**, confirm the ORC-001 freshness and freeze gates are live.
5. **Cross-chain** stays parked; close XC-001 before un-gating it.
6. **Reconcile the redemption-fee LP-replay divergence** and add `cargo check --features self_check` to CI as a dedicated follow-up.

Scope and limitations

In scope at [e49ed10](#) : all eight production canisters' Rust source, the cross-chain module (differential, dev-gated), and the vault-frontend service layer. Out of scope: external and pulled canisters (ICP ledger, XRC, Internet Identity, EVM/SOL RPC internals); test-only and mock canisters; cryptographic primitives; the DFINITY IC fork at rev [fc278709](#) (assumed to match upstream); economic and peg-stability assumptions; and real mainnet state beyond the live deployment-state measurement above. This review is of source code at the named commit, verified in PocketIC and unit tests; it does not substitute for a continuous human audit at materially higher TVL.

How to verify this independently

Every finding, fix, and test fence is traceable through the public repository

[RumiLabsXYZ/rumi-protocol-v2](#) :

- The machine-readable findings list, prior-finding status table, coverage cross-check, and measured deployment state live alongside this report in [audit-reports/2026-06-09-e49ed10/](#) .
- The fixes reproduce with `cargo test -p rumi_protocol_backend --lib` and `POCKET_IC_BIN=./pocket-ic cargo test -p rumi_protocol_backend --test pocket_ic_tests` , plus the new fences [audit_pocs_2026_06_09_redemption_userop_locks](#) (backend), [audit_pocs_ic_s_001_ar_s_002_icrc_004](#) (stability pool), [audit_2026_06_09_fixes](#) (3pool), and the frontend specs under [src/vault_frontend/.../protocol/](#) .

- Mainnet module hashes are verifiable with `icp canister status <id> --network ic`.
- Responsible-disclosure contact is in `SECURITY.md`.

Rumi Protocol team, 2026-06-09